

# SPANS-03: Trace Analysis & Troubleshooting

---

**Series:** SPANS — Distributed Tracing and Spans | **Notebook:** 3 of 8 | **Created:** December 2025 | **Last Updated:** 04/25/2026

## Root Cause Analysis with Distributed Traces

---

This notebook teaches systematic approaches to troubleshoot issues using span data. You'll learn to identify error patterns, analyze latency, and trace problems to their root cause.

---

## Table of Contents

---

1. [Finding Error Spans](#)
  2. [Error Pattern Analysis](#)
  3. [Latency Analysis](#)
  4. [Finding Slow Requests](#)
  5. [Reconstructing Complete Traces](#)
  6. [Identifying Root Cause](#)
  7. [Downstream Dependency Analysis](#)
  8. [Database Query Troubleshooting](#)
- 

## Prerequisites

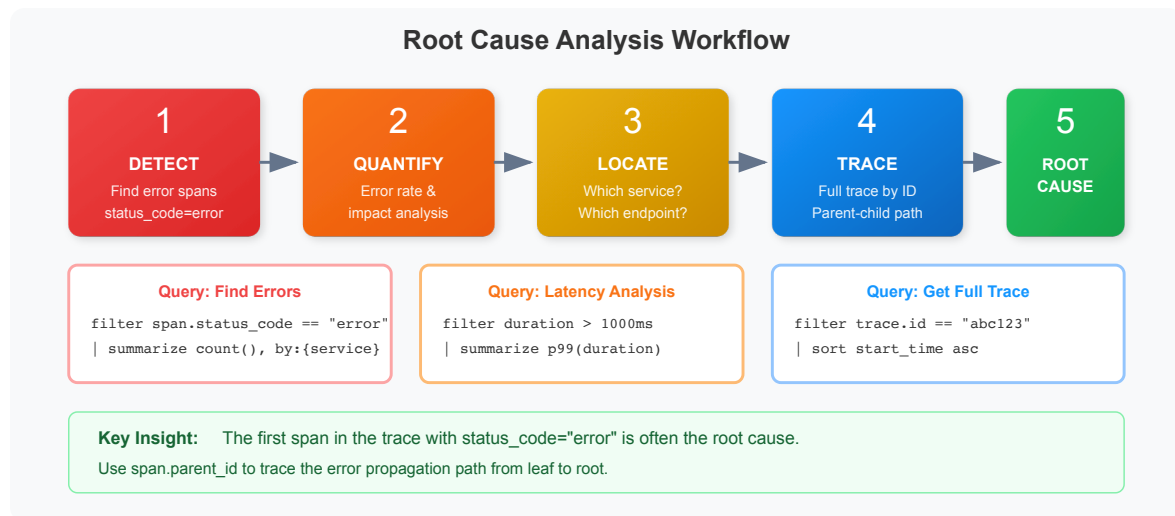
---

Before starting this notebook, ensure you have:

- ☒ Completed **SPANS-01** and **SPANS-02**
- ☒ Access to a Dynatrace environment with span data
- ☒ Understanding of DQL filtering and aggregation

## 1. RCA Workflow Overview

Follow this systematic approach for root cause analysis:



## Key Questions to Answer

| Question            | Query Strategy                                  |
|---------------------|-------------------------------------------------|
| What's failing?     | Filter <code>span.status_code == "error"</code> |
| Where did it start? | Find first error in trace timeline              |
| What's slow?        | Filter <code>duration &gt; threshold</code>     |
| Is it widespread?   | Aggregate by service/operation                  |
| What changed?       | Compare time windows                            |

## 2. Finding Error Spans

Start by identifying error spans in your system:

⚠ Remember: `span.status_code` values are lowercase ( `"error"` , not `"ERROR"` )

```
// Find recent error spans
fetch spans, from:-1h
| filter span.status_code == "error"
| fields start_time,
        service.name,
        span.name,
        span.status_message,
        trace.id,
        duration
| sort start_time desc
| limit 50
```

```
// Count errors by service to see which services are most affected
fetch spans, from:-1h
| filter span.status_code == "error"
| summarize {
    error_count = count(),
    affected_traces = countDistinct(trace.id)
}, by: {service.name}
| sort error_count desc
| limit 20
```

```
// Error rate per service (server spans only)
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {
    total_requests = count(),
    error_count = countIf(span.status_code == "error")
}, by: {service.name}
| fieldsAdd error_rate_pct = (error_count * 100.0) / total_requests
| sort error_rate_pct desc
| limit 20
```

---

### 3. Error Pattern Analysis

---

Group errors to identify common patterns:

```
// Group errors by type and service using collectDistinct
fetch spans, from:-1h
| filter span.status_code == "error"
| summarize {
    occurrence = count(),
    affected_traces = countDistinct(trace.id),
    services_affected = collectDistinct(service.name)
}, by: {span.name, span.status_message}
| sort occurrence desc
| limit 20
```

```
// Find traces with multiple errors (cascading failures)
fetch spans, from:-1h
| filter span.status_code == "error"
| summarize {
    error_count = count(),
    services_affected = collectDistinct(service.name),
    first_error = takeFirst(span.name)
}, by: {trace.id}
| filter error_count > 1
| sort error_count desc
| limit 20
```

```
// Error timeline - visualize errors over time
fetch spans, from:-24h
| filter span.kind == "server"
| makeTimeseries {
    errors = countIf(span.status_code == "error"),
    total = count()
}, interval: 5m, by: {service.name}
```

---

## 4. Latency Analysis

---

Analyze latency patterns to find performance issues:

## Latency Percentile Guide

### p50

Median  
Typical user  
experience

### p95

1 in 20  
5% of users see  
this or worse

### p99

1 in 100  
1% of users see  
this or worse

### max

Worst case  
May include  
outliers

#### DQL Example:

```
fetch spans | filter span.kind == "server"  
| summarize p50=percentile(duration,50), p99=percentile(duration,99) by:{service.name}
```

```
// Latency percentiles by service  
fetch spans, from:-1h  
| filter span.kind == "server"  
| summarize {  
    requests = count(),  
    p50_ms = percentile(duration, 50) / 1000000,  
    p95_ms = percentile(duration, 95) / 1000000,  
    p99_ms = percentile(duration, 99) / 1000000,  
    max_ms = max(duration) / 1000000  
}, by: {service.name}  
| sort p99_ms desc  
| limit 20
```

```
// Latency percentiles by operation  
fetch spans, from:-1h  
| filter span.kind == "server"  
| summarize {  
    requests = count(),  
    p50_ms = percentile(duration, 50) / 1000000,  
    p95_ms = percentile(duration, 95) / 1000000,  
    p99_ms = percentile(duration, 99) / 1000000  
}, by: {service.name, span.name}  
| filter requests > 10  
| sort p95_ms desc  
| limit 30
```

```
// Latency trend over time (use bin() for percentiles since makeTimeseries
doesn't support percentile)
fetch spans, from:-24h
| filter span.kind == "server"
| fieldsAdd time_bucket = bin(start_time, 10m)
| summarize {
    p95_ms = percentile(duration, 95) / 1000000,
    request_count = count()
}, by: {time_bucket, service.name}
| sort time_bucket desc, service.name
| limit 100
```

---

## 5. Finding Slow Requests

---

Identify and analyze slow requests:

```
// Find slow server spans (> 1 second)
fetch spans, from:-1h
| filter span.kind == "server"
| filter duration > 1s
| fieldsAdd duration_ms = duration / 1000000
| fields start_time,
    service.name,
    span.name,
    duration_ms,
    trace.id
| sort duration_ms desc
| limit 50
```

```
// Find slow traces with summary of services involved
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {
    trace_duration_ms = max(duration) / 1000000,
    span_count = count(),
    entry_point = takeFirst(span.name),
    services = collectDistinct(service.name)
}, by: {trace.id}
| filter trace_duration_ms > 1000
| sort trace_duration_ms desc
| limit 20
```

```
// Identify slow operations (candidates for optimization)
fetch spans, from:-1h
| filter span.kind == "server"
| filter duration > 500ms
| summarize {
    slow_count = count(),
    avg_duration_ms = avg(duration) / 1000000,
    max_duration_ms = max(duration) / 1000000
}, by: {service.name, span.name}
| sort slow_count desc
| limit 20
```

---

## 6. Reconstructing Complete Traces

---

Once you've identified a problematic trace, reconstruct the full picture:

```
// Get a sample trace ID from error spans
fetch spans, from:-1h
| filter span.status_code == "error"
| fields trace.id, service.name, span.name
| limit 5
```

```
// Reconstruct full trace (replace YOUR_TRACE_ID with actual trace.id)
fetch spans, from:-1h
// | filter trace.id == "YOUR_TRACE_ID"
| fieldsAdd duration_ms = duration / 1000000
| fields start_time,
    service.name,
    span.name,
    span.kind,
    duration_ms,
    span.status_code,
    span.parent_id,
    span.id
| sort start_time asc
| limit 100
```

```
// Analyze trace complexity
fetch spans, from:-1h
| summarize {
    span_count = count(),
    services_involved = countDistinct(service.name),
    has_errors = countIf(span.status_code == "error") > 0,
    total_duration_ms = sum(duration) / 1000000
}, by: {trace.id}
| sort span_count desc
| limit 20
```

## 7. Identifying Root Cause

Use these patterns to find the origin of problems:

### Root Cause Identification Checklist

RCA

- ☐ Find the FIRST error in the trace timeline `min(start_time)`
- ☐ Check if error propagated from a downstream service `span.kind=="client"`
- ☐ Identify the SLOWEST span contributing to latency `max(duration)`
- ☐ Look for database queries or external calls `isNotNull(db.system)`
- ☐ Check for retry patterns (repeated similar spans) `count() by: span.name`



```
// Find the FIRST error span in failing traces
fetch spans, from:-1h
| filter span.status_code == "error"
| summarize {
    first_error_time = min(start_time),
    first_error_service = takeFirst(service.name),
    first_error_span = takeFirst(span.name),
    error_description = takeFirst(span.status_message)
}, by: {trace.id}
| sort first_error_time desc
| limit 20
```

```
// Find the bottleneck span in each trace
fetch spans, from:-1h
| summarize {
    max_duration_ms = max(duration) / 1000000,
    total_spans = count(),
    services = collectDistinct(service.name)
}, by: {trace.id}
| filter max_duration_ms > 500
| sort max_duration_ms desc
| limit 20
```

```
// Time spent per span kind (where is time going?)
fetch spans, from:-1h
| summarize {
    total_time_ms = sum(duration) / 1000000,
    span_count = count(),
    avg_time_ms = avg(duration) / 1000000
}, by: {span.kind}
| sort total_time_ms desc
```

---

## 8. Downstream Dependency Analysis

---

Analyze failures in downstream services (CLIENT spans):

```
// Find which downstream services are causing errors
fetch spans, from:-1h
| filter span.kind == "client" and span.status_code == "error"
| summarize {
    failure_count = count(),
    sample_error = takeFirst(span.status_message),
    affected_traces = countDistinct(trace.id)
}, by: {service.name, span.name}
| sort failure_count desc
| limit 20
```

```
// Dependency health – error rates for outbound calls
fetch spans, from:-1h
| filter span.kind == "client"
| summarize {
    calls = count(),
    errors = countIf(span.status_code == "error"),
    p95_latency_ms = percentile(duration, 95) / 1000000
}, by: {service.name, span.name}
| fieldsAdd error_rate_pct = (errors * 100.0) / calls
| filter error_rate_pct > 5 or p95_latency_ms > 500
| sort error_rate_pct desc
| limit 20
```

```
// Map service-to-service dependencies
fetch spans, from:-1h
| filter span.kind == "client"
| filter isNotNull(server.address)
| summarize {
    call_count = count(),
    error_count = countIf(span.status_code == "error"),
    avg_latency_ms = avg(duration) / 1000000
}, by: {service.name, server.address}
| fieldsAdd error_rate_pct = (error_count * 100.0) / call_count
| sort call_count desc
| limit 30
```

---

## 9. Database Query Troubleshooting

---

Analyze database operations for performance issues:

```
// Find slow database queries
fetch spans, from:-1h
| filter isNotNull(db.system)
| filter duration > 100ms
| summarize {
    query_count = count(),
    avg_ms = avg(duration) / 1000000,
    p95_ms = percentile(duration, 95) / 1000000,
    max_ms = max(duration) / 1000000
}, by: {db.system, db.name, span.name}
| sort p95_ms desc
| limit 20
```

```
// Find failing database operations
fetch spans, from:-1h
| filter isNotNull(db.system) and span.status_code == "error"
| summarize {
    error_count = count(),
    sample_error = takeFirst(span.status_message)
}, by: {db.system, db.name, span.name}
| sort error_count desc
| limit 20
```

```
// Quick service health check (use for dashboards)
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {
    requests = count(),
    errors = countIf(span.status_code == "error"),
    p50_ms = percentile(duration, 50) / 1000000,
    p99_ms = percentile(duration, 99) / 1000000
}, by: {service.name}
| fieldsAdd error_rate_pct = (errors * 100.0) / requests
| sort error_rate_pct desc
| limit 20
```

---

## Summary

---

In this notebook, you learned:

- ✓ **RCA Workflow** - Systematic approach: Detect → Isolate → Analyze → Correlate → Resolve
  - ✓ **Find errors** using `span.status_code == "error"` and count affected traces
  - ✓ **Error patterns** with `collectDistinct()` to see affected services
  - ✓ **Latency analysis** using percentiles (p50, p95, p99) and `bin()` for trends
  - ✓ **Slow request analysis** to identify optimization candidates
  - ✓ **Trace reconstruction** to see the full picture
  - ✓ **Root cause identification** - first error, slowest span, bottlenecks
  - ✓ **Dependency analysis** - CLIENT spans show downstream failures
  - ✓ **Database troubleshooting** for slow or failing queries
- 

## Next Steps

---

Continue to **SPANS-04: Service Dependencies & Flow Analysis** to learn:

- Mapping service-to-service relationships
  - Analyzing async messaging patterns
  - Visualizing request flows
  - Critical path analysis
- 

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*